

ТЕМА: РЕКУРСИЯ И СТЕК



ВВЕДЕНИЕ

В программировании невозможно обойтись без 3 вещей: ошибок, рекурсии и стека. Здесь мы не будем допускать ошибок, но изучим что такое рекурсия и как с ним связан стек

УРОК: СТЕК

Стек

это структура данных, работающая по принципу "последний пришёл — первый вышел" (LIFO). Используется для хранения промежуточных данных и управления рекурсией



Основные операции

Push

Добавление элемента наверх стека

Pop

Удаление и возврат верхнего элемента стека

Peek

Просмотр верхнего элемента стека без его удаления

IsEmpty

Проверка, пуст ли стек

Пример реализации стека

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
```

```
    else:
        raise IndexError("Pop from an empty stack")

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Peek from an empty stack")

    def is_empty(self):
        return len(self.stack) == 0
```

```
# Пример использования:
s = Stack()
s.push(1)
s.push(2)
print(s.pop()) # Вывод: 2
print(s.peek()) # Вывод: 1
```

Применение

Управление рекурсией
хранение контекста
вызова функций

Операции на строках
обратный порядок
символов

Анализ синтаксиса
проверка
правильности скобок

Преимущества и недостатки

Преимущества

Простота реализации и полезность в различных задачах



Недостатки

Ограниченная память для хранения данных (если стек переполнен)



ВЫВОД

Мы рассмотрели основные концепции рекурсии. Эта тема является основой для понимания более сложных алгоритмов и структур данных, которые используются в программировании